

WEEK 10 · STATE, RENDER, ONE SOURCE OF TRUTH

ONE PLACE WHERE THINGS ARE TRUE

Last week the page started answering. Today it stops holding opinions of its own.

WHERE WE ARE

Week 10 of 12. An application that works, and a bug in it that cannot be fixed by fixing it.

THE NEXT 180 MINUTES

Four cycles, you typing in every one: three layers · a handler that describes · one derivation · a filter from Claude. Break at 85, the surgery at 120, the assignment at 160.

WHAT THIS UNLOCKS

Every project in this room ends up the same shape — which is the only reason the week-13 exam can be the same for all of you.

**This is the week the course
names its architecture.**

TARGET · MINUTE 0

THIS IS WHERE YOU WILL BE AT MINUTE 180

the pantry, taken apart into three layers – it looks exactly like the one you start from

ט
reload

open

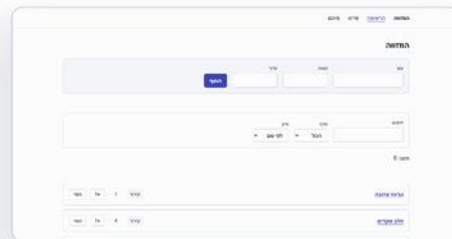
המזווה הרשימה פריט סיכום

המזווה

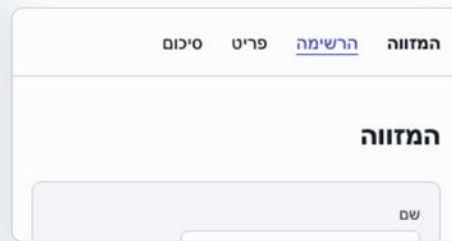
שם	כמות	מדף	הוסף

חיפוש	מדף	מיון
	הכול	לפי שם

מוצג: 6



targets/week-10.png · 1280



375 · the same shape, narrower

► **THE POINT** — the screen does not change today. What changes is that the four faults in the starter **STOP BEING POSSIBLE**: one object, one painter, handlers that only describe. You type each piece once in a cycle first, on a different app.

AI THIS WEEK

PARTNER. IT WRITES; YOU REVIEW IT AGAINST THREE LAWS.

LAYER · WEEKS 8–12 · PARTNER

ALLOWED

Claude writes code — in Claude Code on the folder, or in the chat with the files pasted — **against a rule you wrote into the prompt**, and you read what came back before you run it (cycle 4; cycle 2's undo).

NOT ALLOWED

Code you cannot explain line by line. A refactor that also “fixes” things you did not ask for. **Any AI in cycles 1 and 3’s “you do” and in the surgery** — alone, offline, like week 13.

NO SUBSCRIPTION?

Everything works in the free chat, and nothing in the course depends on a paid plan. Paste the files, copy back what returns, run the same review. Same grade.

CYCLE 1 · 25 MIN

FROM ONE FILE TO THREE LAYERS, AND THE LINE EVERYBODY FORGETS

The class lending library, one file, working — and its first fault: remove a book, add one, and the removed book is back. You move it into `state.js` · `render.js` · `events.js`, wire it in four lines, and the screen ends exactly as it started. Fault included.

1 I DO

10 MIN

3 on the next six slides, 7 in the editor · live-code/01-monolith-to-three-layers · `state.js` with one door · `render.js` · `events.js` · `app.js` in three lines (wrong) · the fourth line

2 WE DO

5 MIN

you move the same code — cut and paste is the point · compare with checkpoint 1 · "document" in `state.js`: zero

3 YOU DO

8 MIN

the second page: `book.html` on the same `app.js`, the id from the address, null is an empty state

4 DEBRIEF

2 MIN

a blank page and a clean console is not a broken render · ↗ reference

YOU DO · AI-FREE

➤ REFERENCE · 5, 6, 8, 9, 20, 21

A BUG YOU CANNOT FIX BY FIXING IT

WHAT HAPPENS

Remove an item. Add a different one. **The removed one is back.**

THE OBVIOUS FIX

The remove handler deletes the row and forgets the array. So make it delete from the array too.

WHY THAT IS NOT A FIX

Now **two** things have to be updated on every removal, by whoever writes the next handler. You have made the bug harder to hit, not impossible.

**Every bug this week is the same bug:
something is true in two places.**

THE ONE-WAY LOOP

```
1  state → render(state) → the screen
2  ↑
3  |_____ events.js ←
```

Data flows one way round. The screen never writes back.

**This has a name in this course,
and we will use it for the rest of
the semester: the one-way loop.**

THREE LAWS, AND EVERYTHING TODAY FOLLOWS FROM THEM

1 · ONE TRUTH

If it is on screen, it came from `state`. If it is not in `state`, it is not real.

2 · ONE PAINTER

Only `render(state)` writes to the screen. Nothing else, ever.

3 · ONE DIRECTION

Handlers change `state`. Nothing reads the screen to find out what is true.

► A fourth follows from all three and it is the one people lose marks on: **ANYTHING DERIVABLE IS DERIVED.**

WHAT EACH LAYER IS FORBIDDEN TO DO

FILE	ITS JOB	WHAT IT MAY NEVER DO
js/state.js	the object, and every derivation of it	touch the DOM
js/render.js	state in, screen out	change state · add listeners · read the screen
js/events.js	turn a user action into a change of state	draw anything
js/app.js	wire the three together, once	grow

► The third column is the useful one. **LAYERS ARE DEFINED BY THEIR PROHIBITIONS**, not by their contents.



ONE DOOR IN, AND IT IS THE REASON EVERYTHING ELSE WORKS

```
1 let state = initialState();
2 const listeners = new Set();
3
4 export function setState(patch) {
5   state = { ...state, ...patch };
6   for (const listener of listeners) listener(state);
7 }
8
9 export const getState = () => state;
```

► Because **NOTHING OUTSIDE THIS FILE ASSIGNS TO STATE**, “did anybody change the collection without redrawing?” has a permanent answer of no. `subscribe(listener)` adds a listener to that set and returns `() => listeners.delete(listener)`: a function remembers the variables around where it was written, which is how the returned function still knows which listener to remove.

THE WHOLE APPLICATION, IN FOUR LINES

```
1 seed(SEED_ITEMS);      // data  -> state
2 subscribe(render);      // state -> screen, on every change, forever
3 wire();                 // screen -> state, via handlers that only setState
4 render(getState());      // draw once, because subscribing does not fire
```

Line 4 is the one people forget, and the page it produces looks exactly like a broken render function.

ONE RENDER, TWO PAGES, AND AN ID THAT IS NOT THERE

```
1 function renderBook(state) {  
2   const mount = document.querySelector('#book');  
3   if (!mount) return;           // not this page: do nothing  
4   const id = new URLSearchParams(location.search).get('id');  
5   const book = selectOne(state, id); // the book, or null  
6   if (book === null) {  
7     // an empty state: a sentence, and a way back  
8   }  
9 }
```

► `get('id')` is **NULL** when the address has no `?id=`, and `selectOne` is `null` when no book has it. **BOTH ARE AN EMPTY STATE** — not an error, not a blank page.

CHECKPOINT 1

YOUR SCREEN SHOULD LOOK LIKE THIS

The screenshot shows a web application for a library. At the top, there is a header with the text "ספריית ההשאלה" (Library) and "הרשימה" (List). Below the header, the main title is "ספריית ההשאלה של הכיתה" (Classroom Library). The interface includes a form with three input fields labeled "שם הספר" (Book Name), "עיתקים" (Copies), and "מדור" (Category), and a green button labeled "הוסף" (Add). Below the form, there is a section titled "מוצגים: 6" (Items: 6) displaying a list of books. Each book entry consists of a title, a rating (1+), a count, and a category.

מוצגים: 6	המפה והים	אור בחלון	שעון החול	גשר על הירקון	כוכבים בצהריים	שביל החלב
מסע	2	1	3	1	4	2
הספר	1+	הספר	הספר	הספר	הספר	הספר

Compare — the page did not move a pixel, and that is the point. What changed is next to it: `js/state.js` has no `document`, `js/render.js` no `addEventListener`, `js/app.js` is four lines. Remove a book and add one: it still comes back. That is cycle 2.

CYCLE 2 · 25 MIN

THE HANDLER THAT DESCRIBES, BY ID

The fault is still there because the two row handlers were moved as they were: they paint. The obvious fix — delete from the array too — makes a **+1** land on the wrong book and paint a 5 where there was a 1. Then both handlers become one `setState` each, by `data-id`, in a new array.

1 I DO

10 MIN

3 on the next four slides, 7 in the editor · live-code/02-handler-that-describes · `render(getState())` from the console brings a removed row back · `splice` by `data-index` (wrong) · `data-id`, `filter`, `map` · the title links to `book.html?id=`

2 WE DO

5 MIN

you type steps 2–3 · compare with checkpoint 2 · `remove`, `+1` below it, `add` — nothing moves

3 YOU DO

8 MIN

undo: a two-line spec, Claude writes it or you do, you review — the array you replaced, kept whole

4 DEBRIEF

2 MIN

"both" — `setState` and `row.remove()` — passes every screen test · ↗ reference

YOU DO · AI AS PARTNER

➤ REFERENCE · 3, 11–16

THERE ARE EXACTLY TWO WAYS TO KEEP A SCREEN UP TO DATE

PATCH IT

Each handler makes the change the user asked for, in the DOM, and keeps the data in step by hand.

Correct for any one action. Not correct for two.

REDRAW IT

Each handler changes the data. One function draws the whole screen from the data, every time.

Correct for any sequence, because there is no sequence.

Not a trick — giving up an optimisation you were never paid for.

READ THE TWO REMOVE HANDLERS, AND NOTHING ELSE

PATCHED

```
1 function onClick(e) {  
2   const row = e.target.closest('li');  
3   row.remove();  
4   count.textContent = rows();  
5   empty.hidden = rows() > 0;  
6 }
```

Three facts to remember. It forgot the only one that mattered.

REDRAWN

```
1 function onClick(e) {  
2   const row = e.target.closest('li[data-id]');  
3   const keep = (i) => i.id !== row.dataset.id;  
4   setState({ items: items.filter(keep) });  
5 }
```

One fact. There is nothing left to forget.

WHERE A ROW SITS IS NOT WHAT A ROW IS

POSITION

```
1 li.dataset.index = index;  
2 // ...  
3 const i = Number(row.dataset.index);  
4 items[i].n += 1;
```

Correct until the first sort. Then it changes the wrong item and shows you the right one changing.

IDENTITY

```
1 li.dataset.id = item.id;  
2 // ...  
3 const id = row.dataset.id;  
4 setState({  
5   items: items.map((i) =>  
6     i.id === id ? { ...i, n: i.n + 1 } : i),  
7 });
```

Correct in every order, because the id does not move.

A NEW ARRAY, EVERY TIME

```
1  setState({ items: [...items, newItem] });           // add
2  setState({ items: items.filter((i) => i.id !== id) }); // remove
3  setState({ items: items.map((i) => (i.id === id ? { ...i, n: i.n + 1 } : i)) }); // change
```

► `push` works today. It mutates the array **ALREADY INSIDE STATE**, so `setState` is handed the object it already had — and everything that ever compares old to new stops being able to.

UNDO IS THE ARRAY YOU REPLACED

REMOVE

```
1 const books = getState().books;
2 setState({
3   books: books.filter((b) => b.id !== id),
4   undo: { books, title }, // the old array, whole
5 });
```

UNDO

```
1 const { undo } = getState();
2 setState({ books: undo.books, undo: null });
```

► Not a description of what was removed — **THE ARRAY BEFORE THE REMOVAL**. Undo is one assignment, and the book comes back in its place, because `filter` and `map` never touched the old one.

CHECKPOINT 2

YOUR SCREEN SHOULD LOOK LIKE THIS

The screenshot shows a web application for a library. At the top, there is a navigation bar with links for 'ספר' (Book), 'הרשימה' (List), and 'ספריית ההשאלה' (Library). Below this is a section titled 'ספריית ההשאלה של הכיתה' (Classroom Lending Library). It features a search bar with a green 'הוסף' (Add) button and three input fields labeled 'שם הספר' (Book Name), 'עותקים' (Copies), and 'מדור' (Category). Below the search bar, there is a list of books, each with a 'הסר' (Remove) button, a '+1' button, and a number. The books are: 'המפה והים' (The Map and the Sea), 'אור בחלון' (Light in the Window), 'שעון החול' (The Sand Clock), 'גשר על הירקון' (A Bridge over the Yarkon), 'כוכבים בצהריים' (Stars in the Afternoon), and 'שביל החלב' (The Milky Way). Each book entry also has a link to 'book.html?id=...'.

שם הספר	עותקים	מדור	הוסף
המפה והים	2	הסר	+1
אור בחלון	1	הסר	+1
שעון החול	3	הסר	+1
גשר על הירקון	1	הסר	+1
כוכבים בצהריים	4	הסר	+1
שביל החלב	2	הסר	+1

Compare — six rows, every title a link to `book.html?id=...`. Then the sequence: remove, +1 on the row below, add — every number stays where it belongs. In `events.js`: no `row.remove`, no `textContent`, three `setState`.

CYCLE 3 · 25 MIN

SEARCH, FILTER AND ORDER ARE ONE DERIVATION

Three controls, wired to nothing. Not three handlers that each filter: three fields in state and one function, `selectVisible`. An order handler that stores its sorted result loses the next book added.

1 I DO

10 MIN

3 on the next four slides, 7 in the editor · live-code/03-one-derivation · `selectVisible` · `selectGenres` and the `select.value` exception · a six-line order handler (wrong) · order in the derivation

2 WE DO

5 MIN

you type steps 1, 2 and 4 · compare with checkpoint 3 · a genre, then one letter: one row, and the select keeps its genre

3 YOU DO

8 MIN

two empty states, derived: nothing here yet, and nothing matched

4 DEBRIEF

2 MIN

a stored "visible" list works in every demo · ↗ reference

YOU DO · AI-FREE

WHAT BELONGS IN THE OBJECT, AND WHAT DOES NOT

IN

```
1 const state = {  
2   items: [],           // not computable  
3   query: '',           // the user typed  
4   category: 'all',     // the user chose  
5   sort: 'title',       // the user picked  
6   undo: null,          // a past state  
7 };
```

OUT

```
1 const state = {  
2   visibleItems: [],    // = filter + sort  
3   count: 0,            // = its length  
4   categories: [],      // = the shelves  
5   summary: {},         // = page three  
6 };
```

► The test is one question per field: **CAN I COMPUTE IT FROM THE OTHERS?** If yes, it does not live here.

SEARCH, FILTER AND SORT ARE NOT THREE BEHAVIOURS

```
1 export function selectVisible(state) {  
2   return state.items  
3     .filter((item) => item.title.includes(state.query))  
4     .filter((item) => state.category === 'all' || item.category === state.category)  
5     .sort((a, b) =>  
6       state.sort === 'number' ? a.number - b.number : a.title.localeCompare(b.title, 'he'),  
7     );  
8 }
```

**They are one derivation, and
the list on screen is its output.**

THEN THE THREE HANDLERS ARE ONE LINE EACH

```
1 queryField.addEventListener('input', () => setState({ query: queryField.value }));  
2 categorySelect.addEventListener('change', () => setState({ category: categorySelect.value }));  
3 sortSelect.addEventListener('change', () => setState({ sort: sortSelect.value }));
```

► Note what the third one does **NOT** do: it does not sort anything. A sort handler longer than one line is sorting, and sorting is a derivation.

OPTIONS NOBODY WROTE, AND THE ONE EXCEPTION

```
1 function renderGenres(state) {  
2   const select = document.querySelector('#genre');  
3   if (!select) return;  
4   const kept = select.value;           // read before...  
5   select.replaceChildren(allOption(), ...selectGenres(state).map(optionFor));  
6   select.value = kept;                 // ...put back after  
7   if (select.value === '') select.value = 'all';  
8 }
```

THE SAME LIST, COUNTED — THE SUMMARY PAGE'S BREAKDOWN
IS THIS

```
1 const perGenre = selectGenres(state).map((genre) => ({  
2   genre,  
3   count: state.books.filter((book) => book.genre === genre).length,  
4 }));
```

► Remove the last book of a genre and its option disappears — **NOBODY WROTE THAT LINE**. And the only place this week that reads the screen: a `<select>` whose children are replaced forgets its selection.

THE EMPTY STATE IS A PROPERTY OF THE DATA, AND THERE ARE TWO OF THEM

```
1  const visible = selectVisible(state);  
2  empty.hidden = visible.length > 0;  
3  empty.textContent =  
4    state.items.length === 0  
5    ? 'nothing here yet – add your first item'  
6    : 'nothing matches the current search and filter';
```

► “There is nothing here” sends the user to the add form. “Nothing matched” sends them to the search box.

ONE SENTENCE FOR BOTH IS A WRONG SIGN.

CHECKPOINT 3

YOUR SCREEN SHOULD LOOK LIKE THIS

The screenshot shows a web application for a library. At the top, there's a header with "ספריית ההשאלה" (Library) and "הרשימה" (List). Below the header, the main title is "ספריית ההשאלה של הכיתה" (Classroom Library). The search form has three input fields: "שם הספר" (Book name), "עיתקים" (Copies), and "מדור" (Category). A green "הוסף" (Add) button is next to the "שם הספר" field. Below the search form, there's a section for "חיפוש" (Search) with a "לפי שם" (By name) dropdown menu and a "הכול" (All) button. The results section shows a list of books with columns for "סדר" (Order), "מדור" (Category), and "חיפוש" (Search). The books listed are:

סדר	מדור	חיפוש
1	רומן	אור בחלון
1	רומן	גשר על הירקון
2	מסע	המפה והים
4	מדע	נוכבים בצהריים
2	מסע	שביל החלב

Compare — six rows by title, the genre filter with three genres. Then: pick the science genre and type one letter — one row, and the filter still says science; "most copies" and add a book with 3 — it lands in its place. Three handlers, one line each.

BREAK

10 MINUTES

Back at 95. Cycle 4 is the AI cycle — a chat window open, free or paid, and `ai-lab/04-add-a-filter/artifact/start/` in your editor. At 120, the surgery: no AI, no internet, alone.

CYCLE 4 · 25 MIN

ADD A FILTER — AND REVIEW WHAT CAME BACK AGAINST THREE LAWS

Prompt 1 is what almost everyone asks: one file pasted, and "I would rather not touch the others". What returns works when you tick the box — and stores a filtered list in state and hides rows in the handler. You catch it by doing one more thing with the box still ticked.

1 I DO

7 MIN

3 on the next two slides, 4 in the chat · ai-lab/04-add-a-filter · P1 live · paste it · tick: 4 rows · the good journal entry on screen

2 WE DO

6 MIN

box ticked, then +1, search, add — find what breaks before it has a name

3 YOU DO

10 MIN

your own P2 with the three laws written in · review before you run · one journal line

4 DEBRIEF

2 MIN

a stored filtered list looks like a service · ↗ reference

YOU DO · AI AS PARTNER

PROMPT 1, AND WHAT CAME BACK

CLAUDE ADDED A FILTER. FIND THE TWO FAULTS.

THE PROMPT, WORD FOR WORD

I have a list of books on my page, and a checkbox with id="multi": "only books with more than one copy". Add a filter to it – when it is checked, only books with more than one copy are shown. Here is my events.js. I'd rather not touch the other files, they work.

<js/events.js – pasted whole>

THE ANSWER, WRAPPED TO FIT (WHOLE: TRANSCRIPT.MD)

```
1 multi.addEventListener('change', () => {
2   const { books } = getState();
3   // keep the filtered list in state
4   setState({ filtered: multi.checked
5     ? books.filter((b) => b.copies > 1)
6     : books });
7   // hide the rows that don't match
8   const rows = document.querySelectorAll('#books li');
9   for (const row of rows) {
10    const copies = Number(row
11      .querySelector('.book-copies').textContent);
12    row.hidden = multi.checked && copies <= 1;
13  }
14  // #shown = the rows not hidden
15 });
```

► Tick it: four rows, “shown: 4”. Now keep it ticked and press +1 on any book. **WHICH LAW BROKE FIRST?**

CYCLE 4 · ON SCREEN BEFORE ANYONE WRITES

WHAT A JOURNAL ENTRY THAT IS WORTH SOMETHING LOOKS LIKE

WHAT I ASKED · WHAT CAME BACK

The prompt, word for word. What returned, in one line — including what you did not ask for: `filteredShelf`, a toolbar inside `render`, a `debounce`.

WHAT WAS WRONG, AND HOW I SAW IT

A thing you did in the browser — “I rated an album with the filter on and the list came back unfiltered.” Then what you kept, what you deleted, and what you learned about the prompt itself.

► **AI-EXEMPLARS/EXEMPLARS/PROMPT-JOURNAL-GOOD.MD** in your course materials — and it is about today’s fault, in somebody else’s project. “It didn’t work” is not a line.

PROMPT 2 · THE LAWS IN THE PROMPT

WRITE THE RULE INTO THE PROMPT, AND THE ANSWER IS THREE LINES

WHAT P2 ADDS

1. One truth: nothing that can be computed is stored. The checkbox is one boolean field.
2. One painter: the handler touches no element and reads nothing from the screen.
3. One direction: the handler is one `setState`. The filter is one more `.filter` in `selectVisible`.

WHAT CAME BACK

```
1 // state.js
2 multi: false,
3 .filter((book) => !state.multi || book.copies > 1)
4
5 // events.js
6 multi.addEventListener('change', () =>
7   setState({ multi: multi.checked }));
```

► Ticked, then +1 on a one-copy book: **IT APPEARS** — it has two now, and the derivation answers again on every draw. Nobody told the handler about the filter.

CHECKPOINT 4

THREE QUESTIONS, THREE YESES

You ran Prompt 1's answer

1 with the box still ticked

and did one more thing — +1 , a search, an add.

Every one of the three laws is in your P2 as something you can check in the answer, and you checked it

2 by reading, before running

.

Your journal line says

3 what the browser showed

— a number of rows, a value in state.

SURGERY · 120–160

THE MINI-SURGERY, AND WHY THE FORMAT MATTERS MORE THAN THE MARKS

TASK	THE WORDING, AND IT IS WEEK 13'S	MINUTES
1 · repair	"The application does X. It should do Y. Find the cause and repair it."	10
2 · extend	"Add Z. Change no behaviour that already works."	10
3 · restructure	"Move this so that a future change to it is local. Behaviour identical."	10

► Same three verbs in week 13, on **YOUR OWN** project, at three times the size. Today is a rehearsal, and it is **GRADED LIGHTLY ON PURPOSE**.

THIRTY MINUTES ALONE, EIGHT TOGETHER, FIFTEEN ON THE PANTRY

122–152 · ALONE

Mini-surgery. One repair, one addition, one restructure, on an application you have never seen.

No AI, no internet, no neighbour.

Write the minutes on the sheet — finding versus fixing.

152–160 · TOGETHER

The walkthrough, on the projector, straight away. **Bring the sheet you filled in.** Task 1 is cycle 2's mistake; task 2 is cycle 3's.

160–175 · THE ASSIGNMENT

Part one's first step in class — the four sequences, the sheet, the first commit. The rest is the refactor you typed three times today, **at home, before lesson 11.**

► Thirty percent of class assignment 10 is this sheet — **GRADED LIGHTLY, ON PURPOSE.**

ASSIGNMENT · 160-175

CLASS ASSIGNMENT 10: THE SAME MOVE, ON THE PANTRY

1 Part one · the split · cycles 1 and 2

the sheet first, then three layers, four lines of `app.js`, `data-id`, handlers that only `setState`.

2 Part two · derived, not stored · cycle 3

`selectVisible`, `selectCategories`, one-line handlers, two empty states.

Parts three and four · cycles 1 and 2, you do

- 3 `item.html?id=`; undo as a snapshot. The summary is the same move on a third page, and its breakdown is the count on cycle 3's genre slide.

FIFTEEN MINUTES IN CLASS

Run the four sequences in the starter and fill `observations-he.md` **before** you move a line. Commit it. The rest at home.

DIFFERENT NAMES, SAME MOVES

`#books` is `#list`, `selectGenres` is `selectCategories`. Nothing is copied; everything is recognised.

► **DUE BEFORE LESSON 11** — repo URL and SHA in Moodle.

FOUR WAYS THE SAME DISEASE SHOWS UP IN YOUR OWN CODE

- 1 Remove an item, add one —
the removed one returns.
- 2 Search, then add something that does not match —
it appears anyway.
- 3 Search, press +1 on a row, clear the search —
a different row went up.
- 4 Sort, then add —
the sort is gone.

► All four are in the assignment starter's `js/app.js`. All four need **TWO ACTIONS IN THE RIGHT ORDER**, which is why nobody found them while writing it.

THREE QUESTIONS BEFORE YOU GO

- 1 Your state has `items` and `visibleItems`.
Which line of code will eventually be wrong, and who writes it?
- 2 You call `render(state)` twice in a row with the same object.
What must be true of the screen, and what does it prove?
- 3 A sort handler in your code is four lines long.
What is in the other three, and where do they belong?

GO DEEPER

The reference deck for this week — every slide the lecture pointed at, in order.

[2 · Two copies, and the distance between them](#) ← spine

20

[3 · The same application, twice](#) ← spine 13

[4 · “So this is React?”](#) ← spine 37

[5 · A function, not an object literal](#) ← spine 4

[6 · seed\(\) copies — one level deep](#) ← spine 4

[8 · render\(state\) is a function of its argument](#) ← spine 4

[9 · Three things render may never do, and why each one bites](#) ← spine 4

[10 · Why a listener inside render multiplies — and when it would not](#) ← spine 28

[11 · A component is a function: item in, element out](#) ←

spine 13

[12 · The price of a template literal is six lines](#) ← spine 13

[13 · Six lines, and they are the whole price](#) ← spine 13

[14 · When to use which, and it is not a matter of taste](#) ←

spine 13

[15 · Nothing is wrong until something reorders the list](#) ←

spine 13

[16 · One listener, on something render never replaces](#) ←

spine 13

[17 · How long a stored answer stays true](#) ← spine 20

[18 · What select.value does when the option is gone](#) ←

spine 20

[19 · Sorting a copy, and comparing like a person](#) ← spine

20

[20 · The third page has no data, and that is the proof](#) ←

spine 4

[21 · The id in the address: get\(\) and encodeURIComponent\(\)](#) ← spine 4

[22 · Why the split makes every future change local](#) ←

spine 37

[23 · Performance, honestly](#) ← spine 37

[24 · Where a full redraw stops being fine](#) ← spine 37

[25 · The five mistakes, and what each one costs](#) ← spine

37